



INFORME DE EVALUACIÓN PRÁCTICA

I. PORTADA

Tema:	Juego aplicativo: Tower Defense
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 - B
Alumnos participantes:	Altamirano Segovia Jullisa Brigitte Llamuca Abrajan Andrés Joel Romo Nuñez Joseph Alejandro Torosina Armendariz Jeremy Alejandro Tuza Quinatoa Edith Noemí
Asignatura:	Estructura de datos
Docente:	Ing. José Caiza

II. INFORME DE EVALUACIÓN PRÁCTICA

2.1 Objetivos

General:

Desarrollar una aplicación tipo *Tower Defense* utilizando arreglos estáticos, listas doblemente enlazadas y listas circulares para gestionar torres, enemigos y oleadas, trasladando la lógica de Java a Python y creando una interfaz gráfica funcional. .

Específicos:

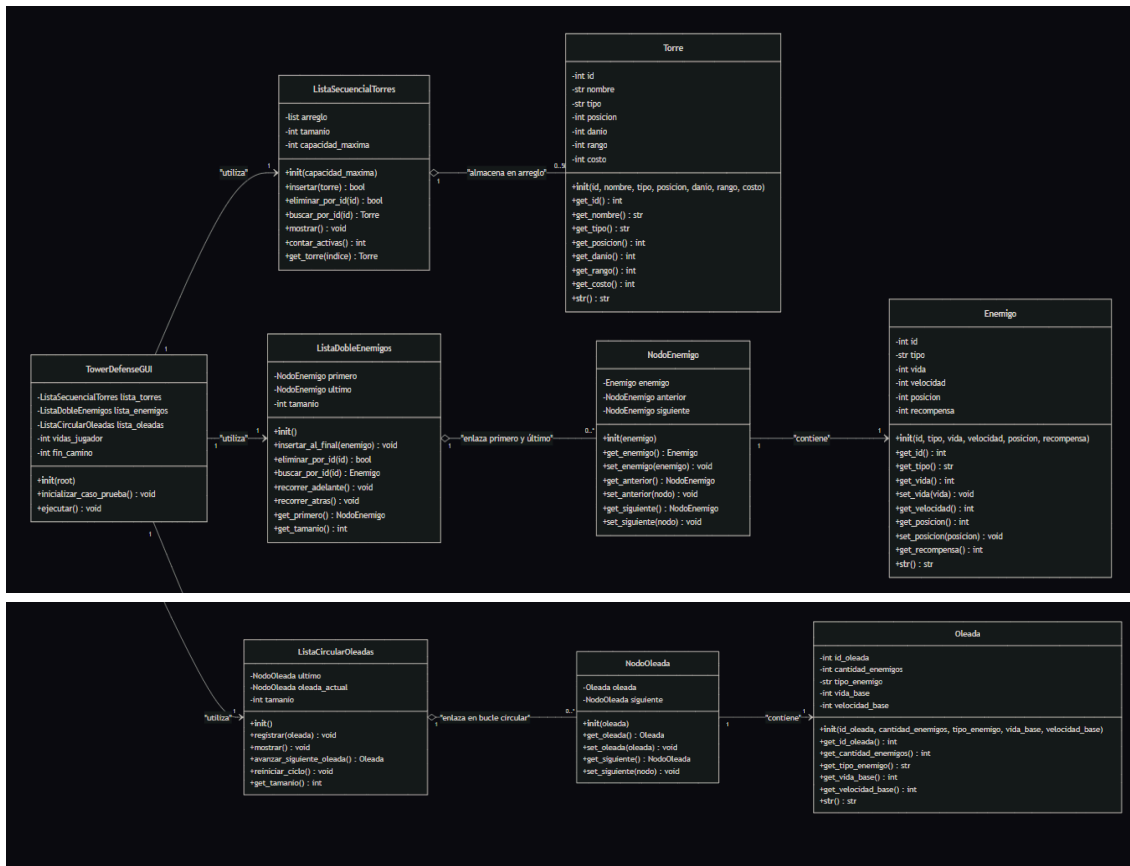
- **Implementar las estructuras de datos** necesarias: arreglo estático para las torres, lista doblemente enlazada para los enemigos y lista circular para las oleadas, traduciendo su funcionamiento de Java a Python.
- **Crear una interfaz gráfica con Tkinter** que permita registrar y eliminar torres, registrar oleadas y avanzar los turnos. También debe mostrar el estado del juego, como el campo de batalla, las estadísticas y las vidas.
- **Agregar una dificultad progresiva**, haciendo que los enemigos aumentan su vida y velocidad con cada oleada, utilizando el recorrido circular de la lista de oleadas.

2.2 Desarrollo

DIAGRAMA DE CLASES



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



TDA

TDA Enemigo

enemigo

Valores:

- id
- tipo
- vida
- velocidad
- posición
- recompensa

Operaciones:

- init__()
- str__()

TDA Oleada

oleada

Valores:

- id_oleada
- cantidad_enemigos
- tipo_enemigo
- vida_base
- velocidad_base

Operaciones:

- init__()
- str__()



TDA Torre

torre

Valores:

- id
- nombre
- tipo
- posicion
- danio
- rango
- costo

Operaciones:

- __init__()
- __str__()

TDA ListaSecuencialTorres

Valores:

- CAPACIDAD_MAXIMA (=50)
- arreglo
- tamaño

Operaciones:

- __init__()
- insertar()
- existe_id()
- eliminar_por_id()
- buscar_por_id()
- mostrar()
- contar_activas()
- get_torre()

TDA NodoEnemigo

Valores:

- enemigo
- anterior
- siguiente

Operaciones:

- __init__()

TDA ListaDobleEnemigos

Valores:

- primero
- ultimo
- tamaño

Operaciones:

- __init__()
- insertar_al_final()



- eliminar_por_id()
- buscar_por_id()
- recorrer_adelante()
- recorrer_atras()
- get_primero()
- get_tamano()

TDA NodoOleada

Valores:

- oleada
- siguiente

Operaciones:

- __init_()

TDA ListaCircularOleadas

Valores:

- Ultimo
- OleadaActual
- Tamano

Operaciones:

- Registrar(oleada)
- ExisteId(idOleada)
- Mostrar()
- AvanzarSiguienteOleada()
- ReiniciarCiclo()
- GetTamano()

Clases

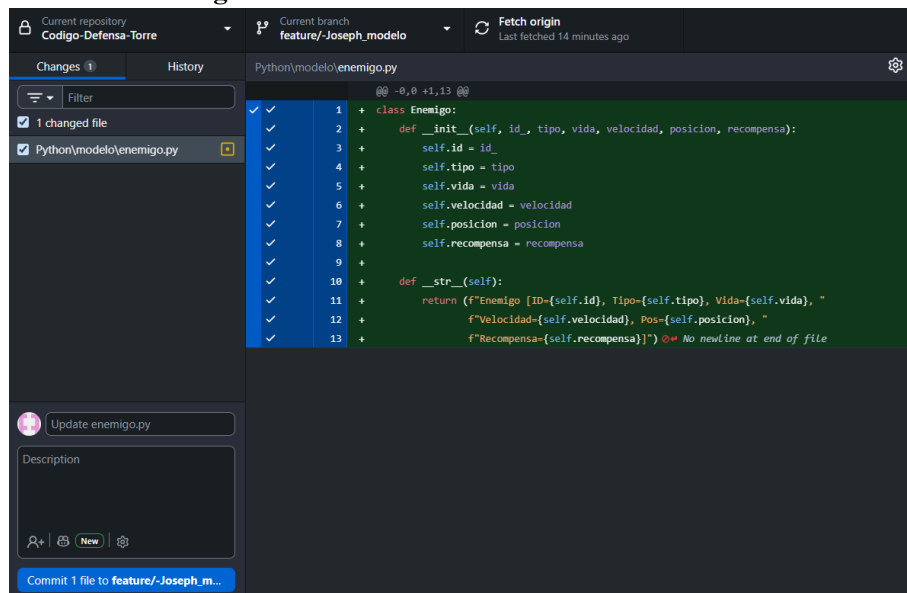


Clase Enemigo

```
enemigo.py M X
Python > modelo > enemigo.py > Enemigo > __str__
1 class Enemigo:
2     def __init__(self, id_, tipo, vida, velocidad, posicion, recompensa):
3         self.id = id_
4         self.tipo = tipo
5         self.vida = vida
6         self.velocidad = velocidad
7         self.posicion = posicion
8         self.recompensa = recompensa
9
10    def __str__(self):
11        return (f"Enemigo [ID={self.id}, Tipo={self.tipo}, Vida={self.vida}, "
12                f"Velocidad={self.velocidad}, Pos={self.posicion}, "
13                f"Recompensa={self.recompensa}"])
```

Representa al enemigo y almacena y muestra la información de cada uno.

Commit de la clase enemigo



Clase Oleada

```
oleada.py U X
Python > modelo > oleada.py > ...
1 class Oleada:
2     def __init__(self, id_oleada, cantidad_enemigos, tipo_enemigo, vida_base, velocidad_base):
3         self.id_oleada = id_oleada
4         self.cantidad_enemigos = cantidad_enemigos
5         self.tipo_enemigo = tipo_enemigo
6         self.vida_base = vida_base
7         self.velocidad_base = velocidad_base
8
9     def __str__(self):
10        return (f"Oleada [ID={self.id_oleada}, Cantidad={self.cantidad_enemigos}, "
11                f"Tipo={self.tipo_enemigo}, VidaBase={self.vida_base}, "
12                f"VelocidadBase={self.velocidad_base}"])
```

Representa a los enemigos que aparecen en cada ronda definiendo sus características.

Commit de la clase oleada



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



The screenshot shows a Git GUI interface. At the top, the repository is 'Codigo-Defensa-Torre' and the current branch is 'feature/-Joseph_modelo'. The file 'Python\modelo\oleada.py' is selected. The left sidebar shows 'Changes' with 1 changed file. The main area displays the code for the 'Oleada' class, which is implemented with 12 lines. The code defines an 'Oleada' class with an '.__init__' method that initializes attributes like 'id_oleada', 'cantidad_enemigos', 'tipo_enemigo', 'vida_base', and 'velocidad_base'. It also includes a '.__str__' method that returns a string representation of the object. The bottom of the interface shows a commit message 'Implementación clase oleada' and a button to 'Commit 1 file to feature/-Joseph m...'.

```
@@ -0,0 +1,12 @@
1 + class Oleada:
2 +     def __init__(self, id_oleada, cantidad_enemigos, tipo_enemigo, vida_base, velocidad_base):
3 +         self.id_oleada = id_oleada
4 +         self.cantidad_enemigos = cantidad_enemigos
5 +         self.tipo_enemigo = tipo_enemigo
6 +         self.vida_base = vida_base
7 +         self.velocidad_base = velocidad_base
8 +
9 +     def __str__(self):
10 +         return (f'Oleada [ID={self.id_oleada}, Cantidad={self.cantidad_enemigos}, "
11 +             f"Tipo={self.tipo_enemigo}, VidaBase={self.vida_base}, "
12 +             f"VelocidadBase={self.velocidad_base}"])
```

Clase Torre

The screenshot shows a code editor with a file named 'torre.py'. The code defines a 'Torre' class with an '.__init__' method that initializes attributes like 'id_', 'nombre', 'tipo', 'posicion', 'danio', 'rango', and 'costo'. It also includes a '.__str__' method that returns a string representation of the object. The code is as follows:

```
1 class Torre:
2     def __init__(self, id_, nombre, tipo, posicion, danio, rango, costo):
3         self.id = id_
4         self.nombre = nombre
5         self.tipo = tipo
6         self.posicion = posicion
7         self.danio = danio
8         self.rango = rango
9         self.costo = costo
10
11     def __str__(self):
12         return (f'Torre [ID={self.id}, Nombre={self.nombre}, Tipo={self.tipo}, "
13             f"Pos={self.posicion}, Daño={self.danio}, Rango={self.rango}, "
14             f"Costo={self.costo}"])
```

Almacena las características de una torre defensiva para saber cómo funcionará.

Commit de la clase torre



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



The screenshot shows a Git web interface for a repository named 'Codigo-Defensa-Torre'. The current branch is 'feature/-Joseph_modelo'. The file 'Python\modelo\torre.py' is shown with 14 lines of Python code. The code defines a class 'Torre' with an '__init__' method that takes parameters: id, nombre, tipo, posicion, danio, rango, and costo. It also has a '__str__' method that returns a string representation of the object. The commit message is 'Implementación de la clase torre'. The commit is titled 'Commit 1 file to feature/-Joseph_m...'.

Clase NodoEnemigo

```
Python > negocio > NodoEnemigo.py > NodoEnemigo > __init__
1 class NodoEnemigo:
2     def __init__(self, enemigo):
3         self.enemigo = enemigo
4         self.anterior = None
5         self.siguiente = None
```

La clase `NodoEnemigo` representa un nodo de una lista doblemente enlazada. En su constructor (`__init__`) recibe un objeto "enemigo" y lo almacena como atributo, e inicializa en `None` los punteros "anterior" y "siguiente", que luego se usan para enlazar cada nodo con el anterior y el siguiente dentro de la lista, permitiendo recorrerla en ambos sentidos.

Commit de la clase `NodoEnemigo`

The screenshot shows a commit message and a code diff for the file 'Python\negocio\NodoEnemigo.py'. The commit message is 'create class NodoEnemigo.py' and 'Implementación de la clase NodoEnemigo como estructura base para una lista doblemente enlazada, incluyendo el almacenamiento del dato y los punteros anterior y siguiente.' The code diff shows the implementation of the 'NodoEnemigo' class, which is identical to the one shown in the previous code block.



Clase ListaSecuencialTorres

```
Python > negocio > ListaSecuencialTorres.py > ...
1  class ListaSecuencialTorres:
2
3      CAPACIDAD_MAXIMA = 50
4
5      def __init__(self):
6          self.arreglo = [None] * self.CAPACIDAD_MAXIMA
7          self.tamano = 0
8
9      def insertar(self, torre):
10
11          if self.tamano >= self.CAPACIDAD_MAXIMA:
12              return False
13          self.arreglo[self.tamano] = torre
14          self.tamano += 1
15          return True
16
17      def existe_id(self, id_):
18          return self.buscar_por_id(id_) is not None
19
20      def eliminar_por_id(self, id_):
21          index = -1
22          for i in range(self.tamano):
23              self.arreglo[i] = self.arreglo[i + 1]
24          self.tamano -= 1
25          self.arreglo[self.tamano] = None
26          return True
27
28      def buscar_por_id(self, id_):
29          for i in range(self.tamano):
30              if self.arreglo[i].id == id_:
31                  return self.arreglo[i]
32          return None
33
34      def mostrar(self):
35          if self.tamano == 0:
36              return ["No hay torres registradas."]
37          return [str(self.arreglo[i]) for i in range(self.tamano)]
38
39      def contar_activas(self):
40          return self.tamano
41
42      def get_torre(self, index):
43          if 0 <= index < self.tamano:
44              return self.arreglo[index]
45          return None
```

La clase ListaSecuencialTorres gestiona un conjunto de torres mediante un arreglo estático de tamaño fijo (50 posiciones), replicando la lógica de la versión original en Java. El método insertar() agrega una torre siempre que no se haya superado la capacidad máxima; existe_id() verifica si ya hay una torre registrada con un id determinado; eliminar_por_id() la localiza y la remueve recorriendo y desplazando el arreglo; buscar_por_id() recorre el arreglo devolviendo la torre cuyo id coincide; mostrar() retorna la representación en texto de todas las torres activas (o un mensaje si no hay ninguna); contar_activas() devuelve cuántas torres hay actualmente registradas; y get_torre() retorna la torre ubicada en una posición (índice) dada.

Commit de la clase ListaSecuencialTorre



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



```
create class ListaSecuencialTorres.py

Implementación de la clase ListaSecuencialTorres para la gestión de torres mediante un arreglo de tamaño fijo (50). Se adaptó la lógica de la versión original en Java,
incluyendo la estructura base y los métodos principales de inserción, búsqueda, eliminación y visualización.

Python/negocio/ListaSecuencialTorres.py

... @@ -0,0 +1,43 @@

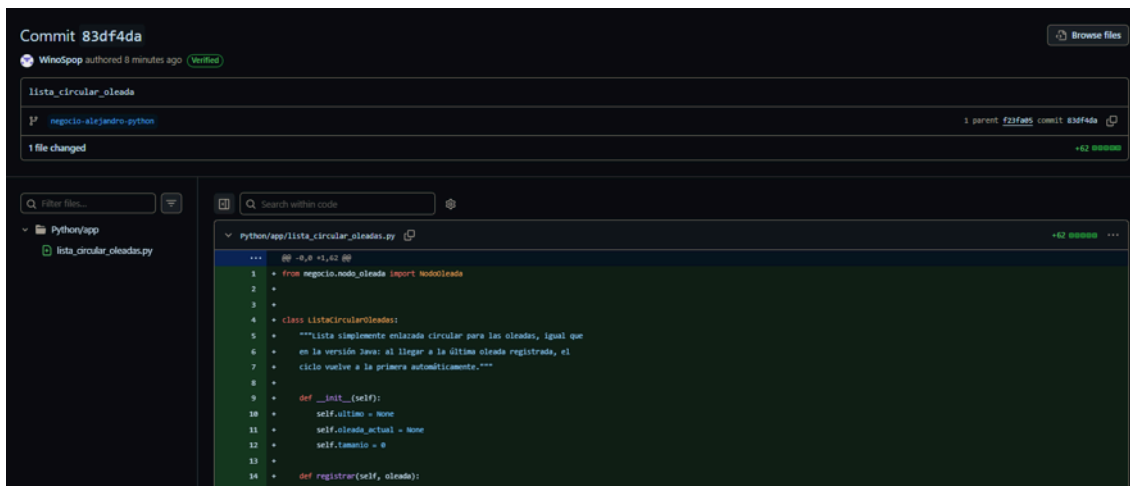
1 + class ListaSecuencialTorres:
2 +     """Arreglo fijo (tamaño máximo definido) para las torres, igual que
3 +     el arreglo estático usado en la versión Java del proyecto."""
4 +
5 +     CAPACIDAD_MAXIMA = 50
6 +
7 +     def __init__(self):
8 +         self.arreglo = [None] * self.CAPACIDAD_MAXIMA
9 +         self.tamano = 0
10 +
11 +     def insertar(self, torre):
12 +         if self.tamano >= self.CAPACIDAD_MAXIMA:
13 +             return False
14 +         self.arreglo[self.tamano] = torre
15 +         self.tamano += 1
16 +         return True
17 +
18 +     def existe_id(self, id_):
19 +         return self.buscar_por_id(id_) is not None
20 +
21 +     def eliminar_por_id(self, id_):
22 +         ...
23 +         return ...
```

Clase ListaCircularOleadas

```
1 from negocio.nodo_oleada import NodoOleada
2
3
4 class ListaCircularOleadas:
5     """Lista simplemente enlazada circular para las oleadas, igual que
6     en la versión Java: al llegar a la última oleada registrada, el
7     ciclo vuelve a la primera automáticamente."""
8
9     def __init__(self):
10         self.ultimo = None
11         self.oleada_actual = None
12         self.tamano = 0
13
14     def registrar(self, oleada):
15         nuevo = NodoOleada(oleada)
16         if self.ultimo is None:
17             self.ultimo = nuevo
18             self.ultimo.siguiente = self.ultimo
19             self.oleada_actual = self.ultimo
20         else:
21             nuevo.siguiente = self.ultimo.siguiente
22             self.ultimo.siguiente = nuevo
23             self.ultimo = nuevo
24             self.tamano += 1
25
26     def existe_id(self, id_oleada):
27         if self.ultimo is None:
28             return False
29         actual = self.ultimo.siguiente
30         while True:
31             if actual.oleada.id_oleada == id_oleada:
32                 return True
33             actual = actual.siguiente
34             if actual is self.ultimo.siguiente:
35                 break
36         return False
37
```

Esta clase almacena las oleadas registradas. Cada nodo apunta al siguiente, y el último nodo apunta de vuelta al primero, formando un ciclo. Mantiene un puntero `oleada_actual` que recorre el ciclo cada vez que se inicia una nueva oleada, de modo que al llegar a la última oleada registrada, el sistema vuelve automáticamente a la primera sin necesidad de reiniciar la lista. Permite registrar oleadas, verificar si un ID ya existe, y listar todas las oleadas registradas.

Commit de la clase ListaCircularOleada



Clase ListaDobleEnemigos

```
1 from negocio.nodo_enemigo import NodoEnemigo
2 You, hace 2 minutos • Agregar lista_doble_enemigos ...
3 class ListaDobleEnemigos:
4     """Lista doblemente enlazada manual para los enemigos activos en
5     el campo, igual que en la versión Java."""
6
7     def __init__(self):
8         self.primeros = None
9         self.ultimo = None
10        self.tamaño = 0
11
12        def insertar_al_final(self, enemigo):
13            nuevo = NodoEnemigo(enemigo)
14            if self.primeros is None:
15                self.primeros = nuevo
16                self.ultimo = nuevo
17            else:
18                self.ultimo.siguiente = nuevo
19                nuevo.anterior = self.ultimo
20                self.ultimo = nuevo
21            self.tamaño += 1
22
23        def eliminar_por_id(self, id_):
24            actual = self.primeros
25            while actual is not None:
26                if actual.enemigo.id == id_:
27                    if actual is self.primeros and actual is self.ultimo:
28                        self.primeros = None
29                        self.ultimo = None
30                    elif actual is self.primeros:
```

La clase ListaDobleEnemigos administra los enemigos mediante una lista doblemente enlazada manual. El método insertar_al_final() agrega elementos actualizando punteros; eliminar_por_id() y buscar_por_id() localizan o remueven un enemigo por su identificador; recorrer_adelante() y recorrer_atras() obtienen la representación en texto en ambas direcciones; y get_primeros() junto con get_tamaño() consultan el estado inicial y la cantidad de elementos.

Commit de la Clase ListaDobleEnemigo



```
Agregar lista_doble_enemigos
Implementa gestion dinamica de enemigos con lista doblemente enlazada manual con atributos primero, ultimo y tamaño.

Python\nnegocio\lista_doble_enemigos.py
@@ -0,0 +1,76 @@

1 + from negocio.nodo_enemigo import NodoEnemigo
2 +
3 + class ListaDobleEnemigos:
4 +     """Lista doblemente enlazada manual para los enemigos activos en
5 +     el campo, igual que en la versión Java."""
6 +
7 +     def __init__(self):
8 +         self.primeros = None
9 +         self.ultimo = None
10 +         self.tamaño = 0
11 +
12 +     def insertar_al_final(self, enemigo):
13 +         nuevo = NodoEnemigo(enemigo)
14 +         if self.primeros is None:
15 +             self.primeros = nuevo
16 +             self.ultimo = nuevo
17 +         else:
18 +             self.ultimo.siguiente = nuevo
19 +             nuevo.anterior = self.ultimo
20 +             self.ultimo = nuevo
21 +             self.tamaño += 1
22 +
23 +     def eliminar_por_id(self, id_):
24 +         actual = self.primeros
25 +         while actual is not None:
```

Clase NodoOleada

```
Python > negocio > nodo_oleada.py > ...
You, hace 5 minutos | 1 autor (You)

1 class NodoOleada:
2     def __init__(self, oleada):
3         self.oleada = oleada
4         self.siguiente = None
5
```

Descripción: La clase `NodoOleada` representa un elemento individual para almacenar información de una oleada dentro de una estructura enlazada. Su método `__init__()` inicializa el nodo asignando el valor recibido en `oleada` y configurando el puntero siguiente en `None`.

Commit de la Clase `NodoOleada`

```
Agregar nodo_oleada
Nodo para la lista circular que gestiona las oleadas del juego. Guarda la oleada y referencia al siguiente.

Python\nnegocio\nodo_oleada.py
@@ -0,0 +1,6 @@

1 + class NodoOleada:
2 +     def __init__(self, oleada):
3 +         self.oleada = oleada
4 +         self.siguiente = None
5 +
6 +
```



(clases GUI)
Clase CampoDeBatallaCanvas

```
class CampoDeBatallaCanvas(tk.Canvas):
    """Equivalente a PathPanel: dibuja la ruta, torres y enemigos (en
    línea recta sobre el camino) y muestra un tooltip al pasar el
    mouse sobre cada elemento."""

    def __init__(self, master, gui, **kwargs):
        super().__init__(master, bg=CANVAS_BG, highlightthickness=0, **kwargs)
        self.gui = gui
        self.posiciones_animadas = None
        self.hitboxes = []
        self.tooltip = None
        self.bind("<Configure>", lambda e: self.dibujar())
        self.bind("<Motion>", self._on_motion)
        self.bind("<Leave>", lambda e: self._ocultar_tooltip())

    def set_posiciones_animadas(self, mapa):
        self.posiciones_animadas = mapa
        self.dibujar()

    def limpiar_animacion(self):
        self.posiciones_animadas = None
        self.dibujar()

    def _map_pos(self, pos, left, right):
        clamped = max(0, min(FIN_CAMINO, pos))
        return left + (clamped / FIN_CAMINO) * (right - left)

    def dibujar(self):
        self.delete("all")
        self.hitboxes = []
        w = self.winfo_width() or 900
        h = self.winfo_height() or 320
        left, right = 80, w - 80
        track_y = h // 2

        self.create_rectangle(0, 0, w, h, fill=CANVAS_BG, outline="")
```

Es el encargado de dibujar el campo de batalla: el camino, la base, las torres y los enemigos, según los datos actuales del juego. También es la gestiona que al pasar el mouse sobre una torre, un enemigo o la base, muestra su información (vida, daño, rango, etc.) en una ventana flotante.

Clase TowerDefenseGUI

```
class TowerDefenseGUI(tk.Tk):
    def __init__(self):
        self._construir_interfaz()
        self._registrar_log("Caso de prueba inicial cargado correctamente.")
        self._refrescar_todo()

    def _inicializar_caso_prueba(self):
        self.lista_torres.insertar(Torre(1, "Arquero", "Físico", 3, 20, 2, 50))
        self.lista_torres.insertar(Torre(2, "Cañón", "Explosivo", 8, 35, 3, 100))
        self.lista_oleadas.registrar(Oleada(1, 3, "Básico", 50, 1))
        self.lista_oleadas.registrar(Oleada(2, 2, "Rápido", 40, 2))

    # =====
    # Construcción de la interfaz (layout según boceto)
    # =====
    def _construir_interfaz(self):
        self._aplicar_estilos_ttk()

        cuerpo = tk.Frame(self, bg=BG_OSCURO)
        cuerpo.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=10, pady=10)

        cuerpo.grid_columnconfigure(0, weight=1) # campo de batalla: se come todo el sobrante
        cuerpo.grid_columnconfigure(1, weight=0) # panel de estadísticas: ancho FIJO (no crece)
        cuerpo.grid_rowconfigure(0, weight=0)
        cuerpo.grid_rowconfigure(1, weight=1)
        cuerpo.grid_rowconfigure(2, weight=0)

        # ---- columna izquierda ----
        self._construir_fila_labels(cuerpo).grid(row=0, column=0, sticky="ew", pady=(0, 6))

        campo_frame = tk.LabelFrame(cuerpo, text="Campo de batalla", bg=BG_OSCURO,
```



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Clase principal de la interfaz, hereda de tk.Tk que es la ventana raíz de la aplicación. Contiene las tres estructuras de datos del juego (ListaSecuencialTorres, ListaDobleEnemigos, ListaCircularOleadas), construye todos los paneles (campo de batalla, tablas de estadísticas, botones), y maneja la lógica del juego: registrar/eliminar torres, registrar e iniciar oleadas, avanzar turnos (movimiento de enemigos, ataques de torres, daño, vidas del jugador), y el escalado de dificultad progresivo por cada ronda.

Commit de las clases GUI

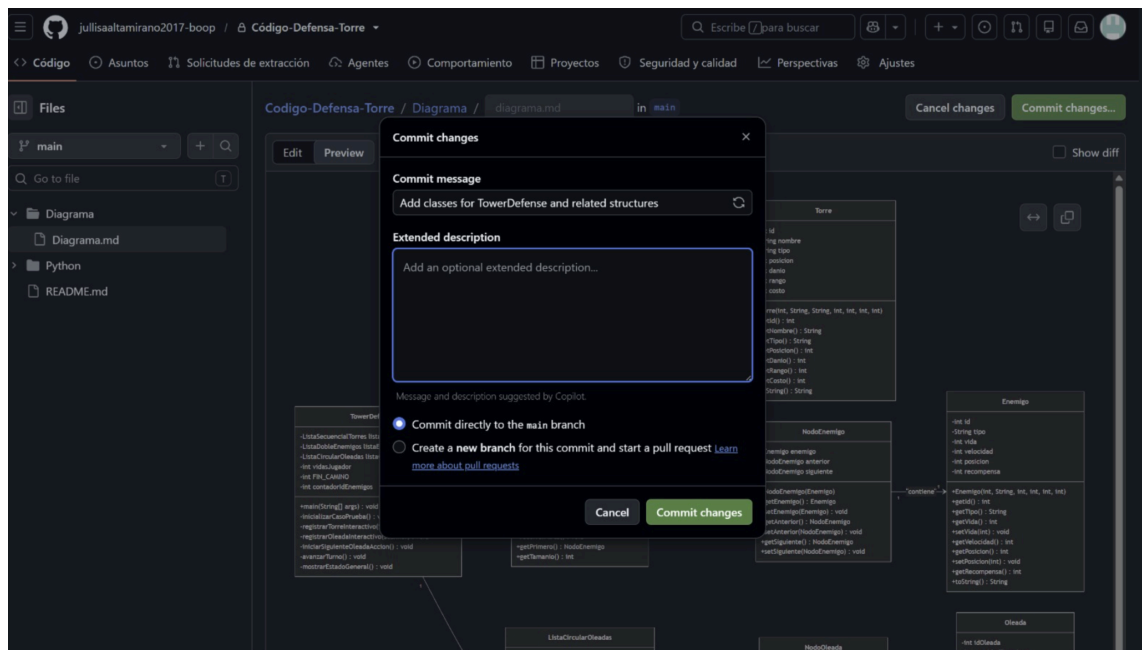
```
... @@ -0,0 +1,799 @@
1 + """
2 + Interfaz gráfica en Python (Tkinter) para el proyecto "Código Defensa
3 + Torre", puerto de app/TowerDefenseGUI.java.
4 +
5 + Cada clase de modelo/negocio vive en su propio archivo (igual que en
6 + el proyecto Java), importada desde los paquetes modelo/ y negocio/.
7 +
8 + Ejecutar con: python3 tower_defense_gui.py
9 + """
10 +
11 + import tkinter as tk
12 + from tkinter import ttk, messagebox
13 +
```

Diagrama

```
1  mermaid
2  classDiagram
3      direction LR
4
5      class Torre {
6          -int id
7          -str nombre
8          -str tipo
9          -int posicion
10         -int danio
11         -int rango
12         -int costo
13         +_init_(id, nombre, tipo, posicion, danio, rango, costo)
14         +get_id() int
15         +get_nombre() str
16         +get_tipo() str
17         +get_posicion() int
18         +get_danio() int
19         +get_rango() int
20         +get_costo() int
21         +_str__() str
22     }
23
24     class Enemigo {
25         -int id
26         -str tipo
```

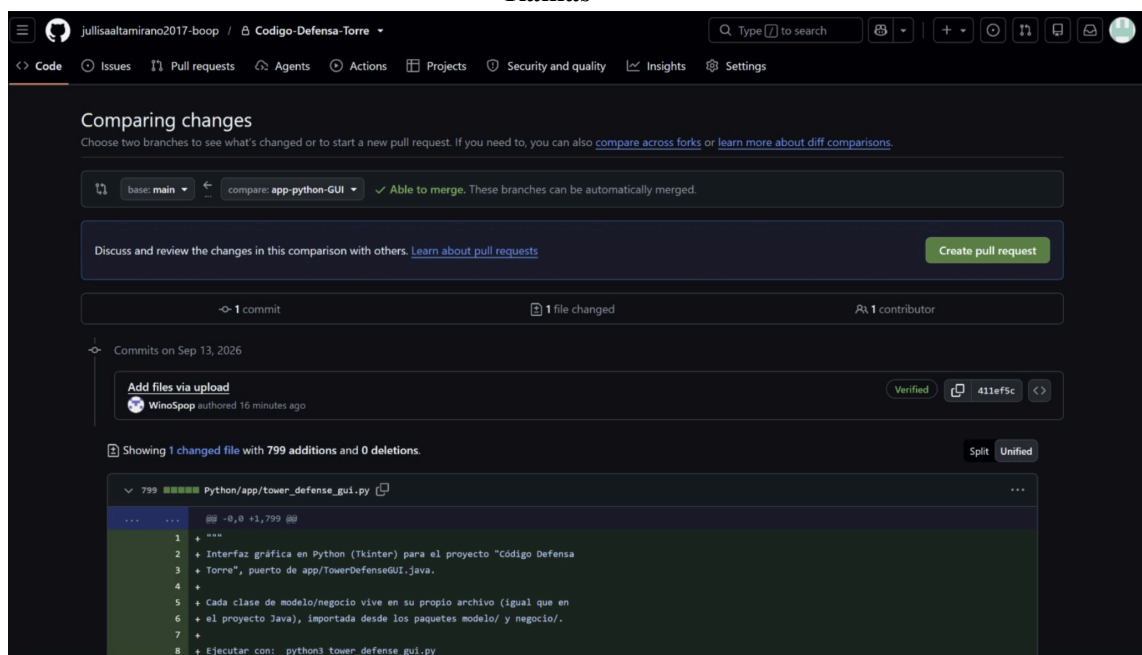


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



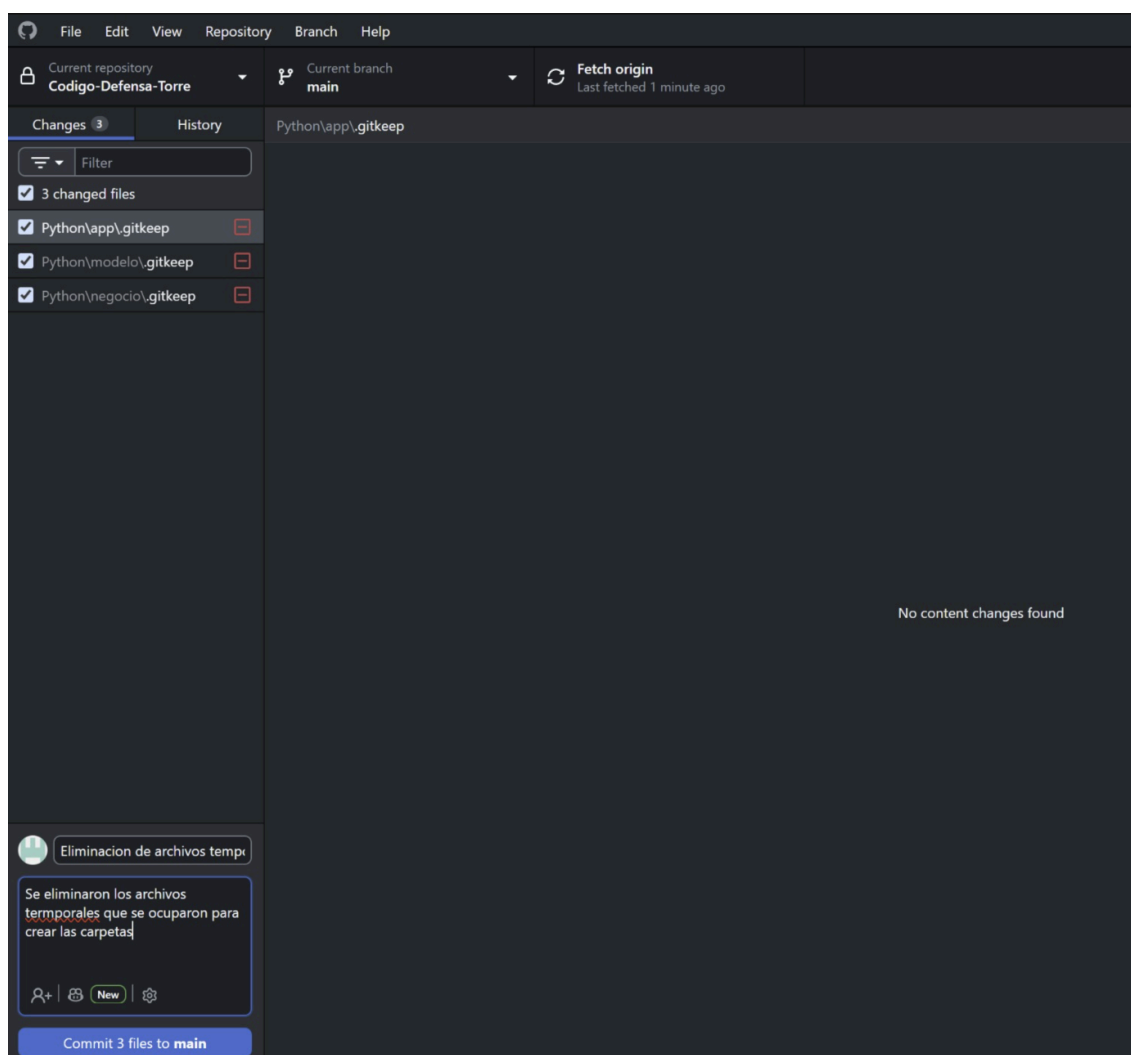
Se realizó el diagrama en base a las clases el cual ha sido construido directamente a partir de la estructura, atributos y métodos ya desarrollados en las clases de Python del proyecto, garantizando así una representación visual precisa y fiel de la implementación real del software.

Ramas



Se integró y fusionó las ramas del trabajo desarrolladas por los integrantes del equipo, uniendo cada rama con la rama principal del repositorio. Este proceso de combinación validó la compatibilidad del código en Python y previene que se dañe la rama principal.

Archivos Temporales



Se eliminaron todos los archivos temporales que se ocuparon para poder crear las carpetas en base a lo que necesitamos.

2.3 Conclusiones

- Estructuras de datos: se comprobó que cada estructura cumple bien su función: el arreglo para las torres, la lista doble para los enemigos y la lista circular para las oleadas.
- Interfaz gráfica: Tkinter permitió crear una interfaz sencilla y funcional para mostrar y controlar el estado del juego.
- Dificultad progresiva: la lista circular permitió manejar las oleadas de forma continua y aumentar la dificultad progresivamente.

2.4 Recomendaciones

- Diseñar el sistema antes de programar: realizar los diagramas UML y definir los TDAs para tener clara la estructura y evitar errores durante la implementación.
- Mantener el código organizado: separar el modelo, la lógica de negocio y la interfaz para facilitar el mantenimiento del proyecto.
- Probar cada módulo por separado: verificar que el arreglo, la lista doblemente enlazada y la lista circular funcionen correctamente antes de integrarlos con la interfaz gráfica.

2.5 Referencias bibliográficas



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



[1] J. Caiza, *Definición de TDA; abstracción; operaciones; pre/postcondiciones; algoritmos con vectores; búsqueda y validación.*, Universidad Técnica de Ambato, 2026.

2.6 Anexos

GitHub: <https://github.com/jullisaaltamirano2017-boop/Codigo-Defensa-Torre>

Actualizaciones o mejoras del Software